

Introduction to OS

Core concept: foundation for modern computing

- run multiple apps
- do complex tasks
- store data reliably

Virtualize; virtual CPU/Mem

Concurrency; run many tasks **seamingly simultaneously**

Persistence; store/get data

Virtualization

- o OS take physical resource turn into virtual form.
eg. turn a CPU into virtually ∞ CPU
- o OS referred as Virtual Machine
- o OS make CPU - Mem - I/O shares so it's called **Resource Manager**
- o OS gives API to app so app can get resources

Processor; What if 2 prog. run same time?

Q.: Which prog. got what & when.

A: Policies, Mem & Files

, API for user prog. to tell desired activities to OS

Memory; Accessed all the time when prog. is running.

- o Prog. keep data structure in mem.
accessed through Load/Store instruction

Q.: When P₁ & P₂ run, it seems value update independently why?

A: - Own private mem. can be mapped to physical

- Mem ref within 1 prog. don't affect add. space
- OS make virtualization of mem.

Concurrency; OS do many things at once. Make problems

Goal; run all prog. with correct semantics

- o What Primitives/Functions needed from OS
- o Mechanism provided by hardware
- o Use them to solve problems. How?
- * Multi-threaded prog. have some problem

But, solution differs from OS-Concurrency as thread share some address space

Persistence; In Mem, data easily lost. DRAM Volatile

→ Power out / System crashed = Data lost

- o Need Disk drive → long live info.
- o OS also manage disk among its tasks = file system
- o Store file **reliably & efficiently** on disk

Q.: Why OS don't make private Virtualized disk for each app.?

- o Files shared across different processes
- o All system call related to file manipulation related to file system
- o OS deals with device drivers get device to do something

Issues; Delay write • RAID for data redundancy & performance improvement

- Handle system crash
- Handle different data structure access method

Build OS consider

- Abstraction convenient & easy
- Performance minimize overhead
- Isolation protect b/w. apps.
- Reliability OS runs non-stop; when fails all apps fail
- Energy efficient longer battery
- Security against malicious
- Mobility smaller devices

History of OS

Early days of Mainframe 1940-1970

Early OS

- o Libraries of commonly used function eg. I/O Manipulation
1 prog. at a time control by Human (Human-Scheduler)

Libraries with Protection

- o Code run on OS' behalf is special, should trust differently!
 - Can't allow prog. access **anywhere** on disk
 - Mode of execution; user, kernel
 - ≠ procedure call, a system call transfer control into OS restoring hardware privilege, Allow full access to hardware of the system
 - o Make I/O Req. o More mem. available to prog.

Multi programming on Mini Comp. 1960-1980

- o Better use of Machine. Focus on Multiprogramming
- o Instead of 1 job at a time, OS load many job into mem, switch between them improve CPU utilization.
- o Issue; Mem protection, Interrupt (Make OS work correctly)
- o UNIX and C-language

Down of PC. 1990 - today

- o DOS; mem. protection Absent
- o Mac OS^{1st}; poor job scheduling
- o Windows; Incorporated feature from minicomputer OS
- o Mac OS X has UNIX at its core

Process & Process API

A Process; a running prog.

CPU Virtualization; OS makes illusion of infinite CPU supply for Processes

Time sharing; taking turn using CPU

To Virtualize CPU, OS needs;

1. Low-level Mechanism; Context switch (start prog) (stop prog)
2. High-level scheduling policy; make decision (who that how long)
3. Mechanism - Policy should separate

Process 4 components

- Program; piece of code process associates with
- Control-state; indicate where to execute next.
- Data-state; hold user data
- Status; ready, suspended, running

Process has

- Memory process can represent Instruction & data
- Register-content execute process; prog-counter
stack pointer, instruction pointer
- I/O list of files process currently has open
Data Segment; Global/Static variable
Stack; Local variable - parameter

Process API

- Create OS creates process to run prog.
- Destroy User kills process
- Wait
- Mix-Control suspend-resume process
- Status how long process run, while state it's in

Process State



perform some Operation make it not ready to run, then suspend

Data Structure; OS is likely to

- Process List; All ready & running process
- Queue of Blocked Processes; I/O done
make up correct process
- Queue of Registered Content; same to mem if process is blocked
- PCB (Process Control Block); Individual structure that stores info about process

API of UNIX

fork(): create new process

Process Identifier (PID) used to name process in UNIX

- Process created by fork(): almost identical to calling process
- Now process has its own copy of address space, register, PC etc.
- OS now see 2 active processes in system
- Child; new process
- Child didn't run at main

wait(): unit for a child to finish

- Force Parents to wait

exec(): Run prog. different from the calling prog.

- Can do executable; word-count etc.
some argument; p.text
- Successful call exec(): never returns.

fork(): & exec(): essential to UNIX shell

name of executable prog + argument passing to UNIX shell

- Figure where executable is
- calls fork(): create new child
- call variant of exec(): run cmd.
- wait for cmd. to complete by wait():
- when child completes, shell returns
- print prompt, ready for next cmd

UNIX are similar, but it use pipe():

Other APIs

- kill(): send signal to process, to sleep-die
- Signal subsystem; deliver external subsystem to process make it receive-process
- Command line tool
• 'ps' list running process
• 'top' display process - how much CPU other resources they using

Virtualization Challenges

- Performance implement without excessive overhead
- Control run efficiently while retaining control over CPU

Limited - Direct execution

- User-mode; any code run = restricted
- kernel-mode; code runs do what it likes
- System Call; user want to access disk
• Execute System Call; Prog. execute trap instruction go into kernel
• Raise Privilege; raise to kernel
• Perform operation

How trap knows which code to run in OS

- kernel set a trap table during boot time
- OS tell what code to run eg. what code to run when hard-disk interrupt
- When use System call cmd (open())
 1. Executed function into C library
 2. Use agree-upon calling which call trap instruction

Switch between Process made by scheduler

Challenges

- Process runs on CPU, OS don't
- OS not running, how it control?
- OS regain control by
 - cooperative mode; wait for system call
 - non-cooperative mode; OS takes control

Cooperative Approach user behaves, use **yield system call** transfer control to OS

- Transfer CPU $\rightarrow$ OS by system call / illegal function
 - Calls to open-read a file
 - Calls to send message to another machine
 - Calls to create new process
 - Calls to yield CPU
 - $\div 0$
 - access mem illegally

◦ If Malicious - bug takes place made it as loop? Reboot.

Non-Cooperative Approach timer interrupt prog. raise it periodically

- When interrupt raised
 1. Running prog. halts.
 2. Pre-config interrupt handler in OS runs
 3. OS regains CPU's control
- Hardware must have enough state of running prog.
- Return-from-trap should be able to resume
- Same as **system-call trap**

Saving - Restore context if need to switch. **low-level code Context switch** will be executed by OS

1. OS save context save register, PC, kernel stack pointer
2. Restore context of soon-to-be-executed
3. OS execute return from trap. Completes

SCHEDULING

scheduling process decides how to commit resources btw. variety of tasks

for computer scheduling = sharing resources across multiple processes

Context switch between process

- low-level mechanism
- high-level mechanism

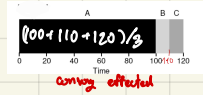
work load; processes running in the system

Scheduling Metrics

turn around time; $T_{\text{turnaround}} = T_{\text{complete}} - T_{\text{arrival}}$

- FIFO first in first out

- A B C arrived at $t=0$ each runs for 10 secs

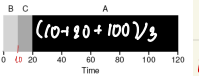


- Drawback;

A B C arrived at $t=0$ A runs for 100 secs, the rest runs 10 secs: Short job behind long job

- SJF (Shortest Job First)

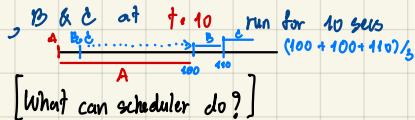
Run shortest one first, then next shortest



Assume all job arrived at $t=0$
BUT real scenario job arrive can vary.

- Job can arrive at anytime not all at once

- A arrived $t=0$ runs for 100 secs



- STCF (Shortest time to completion first)

AKA (PSJF- Preemptive shortest job first)

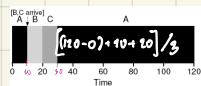
- Timer interrupts and context switching
- Scheduler preempted, decides which job to run
- SJF $\rightarrow$ non-preemptive
- all modern scheduler $\rightarrow$ preemptive

- Anytime new job arrive

- STCF see current vs. new, which one least time left
- It schedule shortest one

- A arrived $t=0$ runs 100 secs

B and C arrived $t=10$ run 10 secs



- Used in Batch scheduling

- Response time metric for interactive performance

Response time; time job arrived to time job scheduled

$T_{\text{response}} = T_{\text{first run}} - T_{\text{arrival}}$

STCF $\rightarrow$ A arrive $t=0$ run 100 secs

B & C arrive $t=10$ run 10 secs

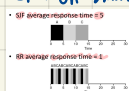


response: $0 + 0 + 10/3 = 3.3...$
NOT GOOD

- Round Robin (RR); Fair. Evenly divide CPU on small time scale

- run job for a time slice
- switch to next job run for a slice

- repeat Note; time slice = length of timer interrupt period



★ SJF risks Starvation! High priority gets ALL

Low priority gets NOTHING

Time slice; shorter = better RR performance

- If too short, context switch cost will beat the gain
- Make time slice long enough to amortize cost of context switching
- Not too long that system become less responsive

Amortization; length of time slice critical to RR.

overhead; fixed cost of operation

Lesser overhead, total cost reduced

ex. time slice 10 ms switch cost 1 ms. Waste 10%.

increase time slice to 100 ms. Waste 1%!

Turn around VS. Response time

- SJF & STCF good turnaround time, bad response time
- RR bad turnaround time, good response time

consider Job with I/O and what if we don't know run time

Incorporating I/O

Job making I/O request doesn't need CPU

- It got blocked waiting I/O done
- Scheduler do other job on CPU

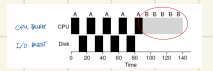
Job complete

- interrupt raised
- OS moves process back to ready state

ex. A B need 50 ms. CPU time

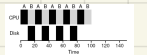
- A run 10 ms, then issue I/O (40 ms.)
- B run 50 ms, no I/O

A first, B later perform bad.



fix: - make each of A 10 ms independent

- system choose to do A or B
- scheduler run processes "interactive" frequently



Q. OS knows little about length of each job

how it do SJF/STCF with no prior knowledge

A. Multi lvl. feedback queue (MLFQ)

Multi level feedback queue (MLFQ)

Goal: Good Turnaround & Response

Concept: - Make Queue with priority level

- Job ready to run single queue at any time.
- MLFQ decides which queue selected
- RR used on job with some queue

MLFQ scheduler set queue priorities

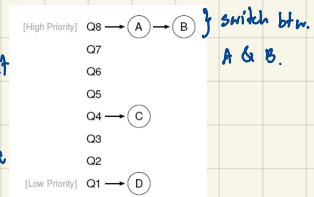
Behavior

- If job wait keyboard input priority = high
- Job use CPU for long time priority = reduced

- MLFQ try to learn about processes as they run use history to predict future

Rules

- Priority $A > B$. A runs
- Priority $A = B$. A & B run RR
- Job just arrived = highest Priority
- Job finished timeslice. Priority = reduced
- Job give up before time slice ends Priority = same



MLFQ Problems

- **Starvation** if many job. long job may not receive share
- Program can trick scheduler to give more share
- Program may change behavior over time
 - CPU bound transition to interactivity
 - With current approach job may not be interactive

Priority Boost

periodically boost priority of all job in system

- Rule (Continue)**
- after period S , all system job to top queue
 - fix starvation
 - CPU bound become interactive scheduler
- get priority boost

What S set to? too high = starvation, too low = interactive share of CPU not enough

- fix rule (4)**
- Once job used up its time even if it give up CPU. Priority = reduced

Final rules

1. Priority $A > B$, A run
2. Priority $A = B$, A B run RR
3. Job arrived in system priority = highest
4. Once job used all allotment time. priority reduce.
5. After S period, all job moved to top queue